Day 19: Storage & StatefulSets

COVER SLIDE / OVERVIEW

```
ubuntu@master-node-dhiraj:~/day19-storage

# Stateful Database Persistence & Volume Binding
$ kubectl create namespace day19-storage
$ kubectl create secret generic pg-secret --from-
literal=POSTGRES_PASSWORD=...
$ kubectl apply -f pg-headless.yaml && kubectl apply -f postgres-
statefulset.yaml
$ kubectl delete pod pg-0 -n day19-storage
Status: PVC Bound & Data Retained After Pod Deletion
```

MISSION ROADMAP (01 TO 07)

- ✓ **01: Inspect Cluster & StorageClass (Task 1)**
- ✓ **02: Namespace & Secret Setup (Task 2)**
- ✓ **03: Headless Service & StatefulSet Deployment (Task 3 & 4)**
- ✓ **04: Dynamic PVC Bound Verification**
- ✓ **05: Record Insertion & Pod Deletion Persistence Test (Task 5)**
- ✓ **06: Storage Chain Deep-Dive & PVC Describe (Task 6-1)**
- ✓ **07: Storage Events & Namespace Teardown (Task 6-2 & 7)**

01 / Storage Architecture & Stateful Workloads

Kubernetes Storage Chain Concepts

- **PersistentVolume (PV):** Cluster-level storage resource provisioned manually or dynamically via a storage provisioner.
- **PersistentVolumeClaim (PVC):** A workload's request for storage (capacity, access modes, storage class).
- **StorageClass:** Defines storage provisioners (e.g., `rancher.io/local-path`) and dynamic volume binding rules.
- **StatefulSet:** Manages stateful pods with unique ordinal indices (`pg-0`), stable network IDs, and sticky storage claims.
- **Headless Service:** `clusterIP: None` service allowing direct DNS resolution to individual Pod identities.
- **Data Persistence:** Pod container filesystems are ephemeral; decoupling storage via PVCs ensures data survives container restarts.

Essential kubectl Storage Commands

- `kubectl get storageclass` — Verify default cluster provisioner
- `kubectl create namespace day19-storage` — Scoped environment
- `kubectl apply -f pg-headless.yaml` — Create headless DNS service
- `kubectl apply -f postgres-statefulset.yaml` — Deploy stateful DB
- `kubectl get pods,pvc -n day19-storage` — Verify storage chain
- `kubectl describe pvc data-pg-0 -n day19-storage` — Inspect claim details

02 / Cluster Inspection & StorageClass Verification (Task 1)

Task 1: Verified cluster storage capabilities, ensuring the default `local-path` StorageClass and provisioner `rancher.io/local-path` are active before deploying workloads.

```
ubuntu@master-node-dhiraj:~$ kubectl get storageclass
NAME         PROVISIONER             RECLAIMPOLICY   VOLUMEBINDINGMODE      ALLOWVOLUMEEXPANSION   AGE
local-path   rancher.io/local-path   Delete          WaitForFirstConsumer   false                  28s
ubuntu@master-node-dhiraj:~$ kubectl get pv
No resources found
ubuntu@master-node-dhiraj:~$ kubectl get pvc -A
No resources found
```

Task 1 Execution Highlights

- Ran `kubectl get storageclass` to identify active provisioner (`rancher.io/local-path`).
- Checked existing persistent volumes and claims across the cluster.
- Confirmed readiness for dynamic volume binding using `local-path`.

03 / Namespace Setup & Lab Secret Provisioning (Task 2)

Task 2: Created the target namespace `day19-storage` and configured a generic Kubernetes Secret `pg-secret` carrying the PostgreSQL administrative password securely.

```
kubectl create namespace day19-storageubuntu@master-node-dhiraj:~$ kubectl create namespace day19-stora
ge
namespace/day19-storage created
ubuntu@master-node-dhiraj:~$ kubectl create secret generic pg-secret --from-literal=POSTGRES_PASSWORD=W
interArcLab123! -n day19-storage
secret/pg-secret created
```

Task 2 Execution Highlights

- Created clean environment: `kubectl create namespace day19-storage`[cite: 4].
- Generated generic secret: `pg-secret` containing key `POSTGRES_PASSWORD` with lab dummy value[cite: 4].
- Secured database initialization parameters without hardcoding plaintext credentials in manifests.

04 / Headless Service & StatefulSet Deployment (Task 3 & 4)

Task 3 & 4: Applied `pg-headless.yaml` and `postgres-statefulset.yaml`, establishing stable network routing and dynamic PVC binding via `local-path`.

```
ubuntu@master-node-dhiraj:~$ kubectl apply -f pg-headless.yaml
service/pg-headless created
ubuntu@master-node-dhiraj:~$ kubectl apply -f postgres-statefulset.yaml
statefulset.apps/pg created
ubuntu@master-node-dhiraj:~$ kubectl get pods,pvc -n day19-storage -w
error: you may only specify a single resource type
ubuntu@master-node-dhiraj:~$ kubectl get pods -n day19-storage -w
NAME   READY   STATUS    RESTARTS   AGE
pg-0   1/1     Running   0          23s
^Cubuntu@master-node-dhiraj:~$ kubectl get pvc -n day19-storage -w
NAME        STATUS   VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS
  VOLUMEATTRIBUTESCLASS   AGE
data-pg-0   Bound    pvc-92a557a9-320b-44f0-884c-63a7f6a5018d   1Gi        RWO            local-path
  <unset>                 50s
^Cubuntu@master-node-dhiraj:~$ kubectl get statefulset -n day19-storage -w
NAME   READY   AGE
pg     1/1     88s
```

```
ubuntu@master-node-dhiraj:~$ kubectl get svc -n day19-storage
NAME          TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)    AGE
pg-headless   ClusterIP   None         <none>        5432/TCP   7s
```

05 / Database Record Creation & Persistence Verification (Task 5)

Task 5: Connected to PostgreSQL via `psql` inside `pg-0`, created `lab_notes` table, inserted test records, deleted the pod, and verified data survival.

```
^Cubuntu@master-node-dhiraj:~kubectl exec -it -n day19-storage pg-0 -- \\
psql -U appuser -d winterarc
psql (16.15)
Type "help" for help.

winterarc=# CREATE TABLE lab_notes (id INT PRIMARY KEY, note TEXT);
INSERT INTO lab_notes VALUES (1, 'data-survives-pod-delete');
CREATE TABLE
winterarc=# INSERT INTO lab_notes VALUES (1, 'data-survives-pod-delete');
INSERT 0 1
winterarc=# SELECT * FROM lab_notes;
 id |           note
----+--------------------------
  1 | data-survives-pod-delete
(1 row)

winterarc=# \q
ubuntu@master-node-dhiraj:~$ kubectl get pvc -n day19-storage
NAME        STATUS   VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS
  VOLUMEATTRIBUTESCLASS   AGE
data-pg-0   Bound    pvc-92a557a9-320b-44f0-884c-63a7f6a5018d   1Gi        RWO            local-path
  <unset>                 4m18s
ubuntu@master-node-dhiraj:~$ kubectl delete pod pg-0 -n day19-storage
pod "pg-0" deleted
ubuntu@master-node-dhiraj:~$ kubectl get pods -n day19-storage -w
NAME   READY   STATUS    RESTARTS   AGE
pg-0   1/1     Running   0          7s
ubuntu@master-node-dhiraj:~$ kubectl exec -it -n day19-storage pg-0 -- psql -U appuser -d winterarc -c
"SELECT * FROM lab_notes;"";"
 id |           note
----+--------------------------
  1 | data-survives-pod-delete
(1 row)
```

06 / Storage Chain Inspection & PVC Describe (Task 6-1)

Task 6-1: Inspected the complete storage hierarchy (StatefulSet, Pods, PVC, PV) and examined detailed binding descriptions for claim `data-pg-0`.

```
ubuntu@master-node-dhiraj:~$ kubectl get statefulset -n day19-storage
NAME   READY   AGE
pg     1/1     6m25s
ubuntu@master-node-dhiraj:~$ kubectl get pvc -n day19-storage
NAME        STATUS   VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS
  VOLUMEATTRIBUTESCLASS   AGE
data-pg-0   Bound    pvc-92a557a9-320b-44f0-884c-63a7f6a5018d   1Gi        RWO            local-path
  <unset>                 6m42s
ubuntu@master-node-dhiraj:~$ kubectl get pv -n day19-storage
NAME                                       CAPACITY   ACCESS MODES   RECLAIM POLICY   STATUS   CLAIM
                STORAGECLASS   VOLUMEATTRIBUTESCLASS   REASON   AGE
pvc-92a557a9-320b-44f0-884c-63a7f6a5018d   1Gi        RWO            Delete           Bound    day19-st
orage/data-pg-0   local-path     <unset>                          6m44s
ubuntu@master-node-dhiraj:~$ kubectl get pods -n day19-storage
NAME   READY   STATUS    RESTARTS   AGE
pg-0   1/1     Running   0          2m37s
ubuntu@master-node-dhiraj:~$ kubectl describe pvc data-pg-0 -n day19-storage
Name:          data-pg-0
Namespace:     day19-storage
StorageClass:  local-path
Status:        Bound
Volume:        pvc-92a557a9-320b-44f0-884c-63a7f6a5018d
Labels:        app=pg
Annotations:   pv.kubernetes.io/bind-completed: yes
               pv.kubernetes.io/bound-by-controller: yes
               volume.beta.kubernetes.io/storage-provisioner: rancher.io/local-path
               volume.kubernetes.io/selected-node: slave-node
               volume.kubernetes.io/storage-provisioner: rancher.io/local-path
Finalizers:    [kubernetes.io/pvc-protection]
Capacity:      1Gi
Access Modes:  RWO
VolumeMode:    Filesystem
Used By:       pg-0
Events:
  Type    Reason                Age                   From
                                                     Message
  ----    ------                ----                  ----
                                                     -------
  Normal  WaitForFirstConsumer  7m20s                 persistentvolume-controller
                                                     waiting for first consumer to be created before bi
nding
  Normal  ExternalProvisioning  7m20s (x2 over 7m20s)  persistentvolume-controller
                                                     Waiting for a volume to be created either by the e
xternal provisioner 'rancher.io/local-path' or manually by the system administrator. If volume creation
 is delayed, please verify that the provisioner is running and correctly registered.
  Normal  Provisioning          7m20s                 rancher.io/local-path_local-path-provisioner-65
d5864f8d-6f84p_024791bf-8342-4f98-bbb7-e20d26433449  External provisioner is provisioning volume for cl
aim "day19-storage/data-pg-0"
  Normal  ProvisioningSucceeded  7m16s                 rancher.io/local-path_local-path-provisioner-65
d5864f8d-6f84p_024791bf-8342-4f98-bbb7-e20d26433449  Successfully provisioned volume pvc-92a557a9-320b-
44f0-884c-63a7f6a5018d
```

07 / Storage Provisioning & Scheduling Events (Task 6-2)

Task 6-2: Analyzed cluster events sorted by timestamp to trace dynamic provisioning steps by `rancher.io/local-path` and pod scheduling execution.

```
ubuntu@master-node-dhiraj:~$ kubectl get events -n day19-storage --sort-by=.lastTimestamp
LAST SEEN   TYPE     REASON                  OBJECT                              MESSAGE
7m26s       Normal   Scheduled               pod/pg-0                            Successfully assigned da
y19-storage/pg-0 to slave-node
3m2s        Normal   Scheduled               pod/pg-0                            Successfully assigned da
y19-storage/pg-0 to slave-node
7m31s       Normal   WaitForFirstConsumer    persistentvolumeclaim/data-pg-0     waiting for first consum
er to be created before binding
7m31s       Normal   ExternalProvisioning    persistentvolumeclaim/data-pg-0     Waiting for a volume to
be created either by the external provisioner 'rancher.io/local-path' or manually by the system adminis
trator. If volume creation is delayed, please verify that the provisioner is running and correctly regi
stered.
7m31s       Normal   Provisioning            persistentvolumeclaim/data-pg-0     External provisioner is
provisioning volume for claim "day19-storage/data-pg-0"
7m31s       Normal   SuccessfulCreate        statefulset/pg                      create Claim data-pg-0 P
od pg-0 in StatefulSet pg success
7m27s       Normal   ProvisioningSucceeded   persistentvolumeclaim/data-pg-0     Successfully provisioned
 volume pvc-92a557a9-320b-44f0-884c-63a7f6a5018d
7m26s       Normal   Pulling                 pod/pg-0                            Pulling image "postgres:
16-alpine"
7m21s       Normal   Started                 pod/pg-0                            Started container postgr
es
7m21s       Normal   Created                 pod/pg-0                            Created container: postg
res
7m21s       Normal   Pulled                  pod/pg-0                            Successfully pulled imag
e "postgres:16-alpine" in 5.25s (5.251s including waiting)
3m4s        Normal   Killing                 pod/pg-0                            Stopping container postg
res
3m3s        Normal   Pulled                  pod/pg-0                            Container image "postgre
s:16-alpine" already present on machine
3m3s        Normal   Created                 pod/pg-0                            Created container: postg
res
3m3s        Normal   Started                 pod/pg-0                            Started container postgr
es
3m3s        Normal   SuccessfulCreate        statefulset/pg                      create Pod pg-0 in State
fulSet pg successful
```

08 / Namespace Teardown & Resource Cleanup (Task 7)

Task 7: Deleted namespace `day19-storage` and verified that cluster storage resources (PVs) were completely garbage collected according to the Delete policy.

```
ubuntu@master-node-dhiraj:~$ kubectl delete namespace day19-storage
namespace "day19-storage" deleted
ubuntu@master-node-dhiraj:~$ kubectl get pv
No resources found
ubuntu@master-node-dhiraj:~$
```

Teardown Summary

- Executed `kubectl delete namespace day19-storage`[cite: 16].
- All associated StatefulSets, Pods, Services, and PVCs were terminated automatically.
- Confirmed `kubectl get pv` returned no leftover cluster volumes, leaving a clean cluster state[cite: 16].

09 / Troubleshooting & Storage Diagnostics Matrix

Symptom / Error	What to Check	Resolution & Key Learning
PVC Pending	StorageClass availability & Events	Ensure StorageClass name matches cluster provisioner (e.g., `local-path` vs `standard`). Check PVC events via `kubectl describe pvc`.
Pod CrashLoopBackOff	Secret keys & environment vars	Verify Secret key name matching (e.g., `POSTGRES_PASSWORD`). Inspect database logs via `kubectl logs pg-0`.
Data missing after recreate	Volume mount directory path	Confirm volume mount path matches PostgreSQL data directory (`/var/lib/postgresql/data`) and PVC is correctly reattached.
PVC remains post Pod delete	StatefulSet retention policy	By design, StatefulSet PVCs are NOT deleted when Pods are deleted to safeguard data against accidental Pod termination.

10 / Deliverable & Architectural Review Checklist

Repository Structure (`day19-storage-statefulsets/`)

- **pg-headless.yaml:** Headless Service definition (`clusterIP: None`).
- **postgres-statefulset.yaml:** PostgreSQL StatefulSet with `volumeClaimTemplates`.
- **README.md:** Detailed documentation, step-by-step commands, storage chain explanation, and Q&A.
- **screenshots/ (task 1 to 7):** Full step-by-step terminal execution evidence.

Engineering Quality Standards

- **Data Integrity:** Proven record survival across pod deletion/recreation cycles.
- **Security Compliance:** No plain-text passwords stored in git manifests.
- **Storage Chain Mastery:** Complete understanding of PV, PVC, StorageClass, and Provisioner interactions.

11 / Interview Readiness & Architecture Q&A

1. PV vs PVC?

- A PV is actual storage provisioned in the cluster; a PVC is a request for storage by a workload that binds to a matching PV.

2. Why dynamic volumeClaimTemplates?

- It automatically provisions individual PVCs per StatefulSet pod replica (e.g. `data-pg-0`), maintaining sticky pod identity.

3. Deployment vs StatefulSet?

- Deployments manage stateless interchangeable pods; StatefulSets manage stateful pods with unique indices and dedicated volumes.

4. Why Headless Service?

- Headless Services (`clusterIP: None`) return individual pod IP addresses via DNS, enabling direct pod-to-pod DB communication.